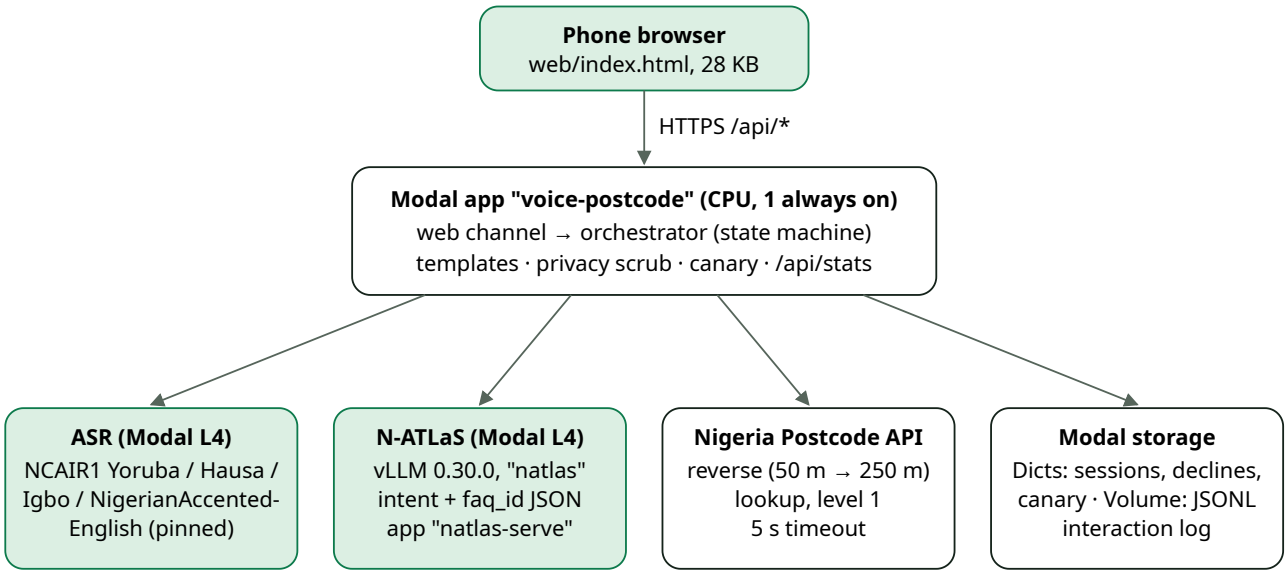
Technical documentation

Voice Postcode Assistant · Team VoxPost NG · Live: https://muideen27--voice-postcode-web.modal.run · Code: https://github.com/Muideen27/voice-postcode-assistant

N-ATLaS is an initiative of the Federal Ministry of Communications, Innovation and Digital Economy, and powered by Awarri Technologies.

1. Architecture



Green boxes are N-ATLaS components. All logic lives in the orchestrator (`app/orchestrator.py`); the web channel only turns HTTP requests into events. Facts come only from the Postcode API and reviewed templates.

2. Components

Path	Role
`web/index.html`	The page (plain HTML/JS, no framework; 28 KB, 9 KB gzip). Matches `web/design/prototype.html`. Tabs: the app and the pilot dashboard.
`web/fonts/`	Self-hosted subset fonts (101 KiB) with a fallback for ẹ ọ ṣ ɓ ɗ ƙ y ị ụ ṅ
`app/main.py`	Modal app: web endpoint, ASR class, canary (every 30 min), unconsented-session sweep
`app/channels/web.py`	HTTP API (section 6)
`app/orchestrator.py`	State machine: language → consent → "did you know" → menu; voice, text, location paths
`app/services/asr.py`	ffmpeg decoding, limits, silence skip (CPU); the four pinned Whisper models (GPU)

Path	Role
`app/services/llm.py`, `prompts.py`	N-ATLaS client, prompts, schema, guard
`app/services/postcode.py`	Postcode API client; auth errors kept apart from "not found"
`app/store.py`, `app/privacy.py`	Hashing, sessions, interaction log, decline and canary counters; PII scrub
`app/stats.py`	Log reading and dashboard numbers
`app/templates/*.json`	Every user-facing string in en, yo, ha, ig
`deploy/natlas_serve.py`	vLLM server for N-ATLaS on an L4

3. Setup from a fresh clone

```
git clone https://github.com/Muideen27/voice-postcode-assistant.git && cd voice-postcode-assistant
python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt
pip install modal && modal setup          # log in to Modal
```

Accept the model terms on Hugging Face for the four `NCAIR1/*` ASR models and `NCAIR1/N-ATLaS`.

Secrets — in the Modal web UI (Secrets → Create new secret → Custom), or with the CLI:

Secret name	Keys
`huggingface`	`HF_TOKEN`
`natlas-api`	`NATLAS_API_KEY` (any long random string; protects the vLLM endpoint)
`voice-postcode`	`POSTCODE_API_KEY` (Nigeria Postcode API key), `HASH_SALT` (`openssl rand -hex 32`)

```
modal secret create voice-postcode POSTCODE_API_KEY=<key> HASH_SALT=$(openssl rand -hex 32)
```

Important: after deleting and re-creating any secret, deploy again. A deployed app stays bound to the old secret and fails to start until it is redeployed.

4. Deploy and operate

```
modal deploy deploy/natlas_serve.py                          # N-ATLaS on vLLM (scale-to-zero)
modal deploy app/main.py                                     # web app + ASR
PILOT_START=2026-10-11T09:00:00+01:00 modal deploy app/main.py   # start counting on the dashboard
scripts/warm.sh 90       # keep LLM + ASR warm for 90 minutes, then back to 0 (Ctrl+C resets)
scripts/warm.sh 17:00    # keep them warm until 17:00 WAT today (up to 12 hours)
scripts/warm.sh off      # back to 0 now
python scripts/export_log.py --pull --since <pilot start>   # docs/validation/interactions.csv + summary.md
python scripts/fill_submission.py && python scripts/build_submission.py   # these PDFs
```

Environment settings (set in `app/main.py` at deploy time):

Name	Values	Effect
`EXPLAIN_MODE`	`faq` (default), `generate`	Explanations from reviewed FAQ templates; `generate` (evals only) uses N-ATLaS text behind the guard
`PILOT_START`	ISO time, e.g. `2026-10-11T09:00:00+01:00`, or empty	The dashboard counts only log rows from this time; empty shows the empty state
`VP_ENV`	`prod` (default), `dev`	`dev` writes log rows to `/logs/dev/`, which no report or dashboard reads
`NATLAS_URL`	URL	The N-ATLaS endpoint

The Postcode API access level is fixed at Lookup level 1 (validity), the level of our key.

5. Tests

```
pytest -q          # 349 unit tests (11 Oct 2026), no network: orchestrator, API client
(recorded responses), ASR (fake GPU),
                   # LLM client (fake transport), web API, privacy scrub, store, stats,
export, templates
pytest -m pilot    # release gate: fails until every Yorùbá/Hausa/Igbo string is native-
speaker reviewed
ruff check .
python evals/natlas_final_test.py      # live intent / FAQ evals (GPU)
modal run evals/asr_smoke.py --clips yo:yo_test.wav
```

6. API contract (all JSON unless noted)

Route	Request	Response
`GET /`	—	the page
`GET /api/strings/<lang>`	—	every template string for `en`, `yo`, `ha` or `ig`
`GET /api/warm`	—	`{"llm": bool, "asr": bool}`; starts the GPUs in the background if cold
`POST /api/language`	`{session, lang}`	Reply (welcome for a new user, else menu)
`POST /api/consent`	`{session, agree}`	Reply (`ask_knew_before` or `declined`)
`POST /api/location`	`{session, lat, lng, accuracy_m}`	Reply (postcode or not found)
`POST /api/location-denied`	`{session}`	Reply (how to allow location)
`POST /api/voice`	multipart `session`, `audio` (≤ 20 s, ≤ 2 MB)	Reply with `transcript`
`POST /api/text`	`{session, text ≤ 500}`	Reply

Route	Request	Response
`POST /api/feedback`	`{session, helpful}` or `{session, knew_before, skipped}`	Reply
`GET /api/stats`	—	pilot dashboard numbers (no transcripts or user IDs)

`session` is a random UUID kept in the browser and hashed by the server before use. A Reply is `{kind, lang, text, postcode, segments, confidence, transcript, distance_m}`.

7. Limits and costs

Item	Value
Voice	≤ 20 s, ≤ 2 MB per recording
Typed text	≤ 500 characters
LLM timeout per turn	20 s (then keyword safety net or "waking up" message)
Postcode API	5 s timeout, one retry after HTTP 429, 600 requests per window
GPU cold start	LLM about 2 minutes (101–349 s measured), ASR 19–76 s
Page load	about 1 s (one CPU web container always on)
Licence cap	fewer than 1,000 active users per 30 days

Costs (Modal list prices checked 11 Oct 2026): L4 GPU about $0.80 per hour while running; both GPUs scale to zero when idle. The always-on CPU web container costs about $0.25 per day. Keeping one GPU warm for an 8-hour session costs about $6.40; both, about $12.80.

8. Licence and attribution

N-ATLaS is an initiative of the Federal Ministry of Communications, Innovation and Digital Economy, and powered by Awarri Technologies. The attribution appears on the language screen, the welcome message, every page footer and these documents. Usage stays under 1,000 active users per 30 days, with no surveillance or profiling. Postcode data comes from the Nigeria Postcode API.